pc端解析GPX文件v1.0

在pc端解析kml文件v1.3版本上进行修改

修改说明

新增属性解析能力

- **原理**：KML 的点信息文本格式为 `<coordinates>lon,lat,alt</coordinates>`，而 GPX 是 `<wpt lat="39.1" lon="116.2">`。
- **修改**：在 `KmlCtx` 结构体中添加了 `attr_buf`（长度 256）。当遇到标签名后面的空格时，状态机进入新的 `STATE_ATTR_COLLECT` 状态，将属性全文收录。当遇到 `>` 时，触发 `parse_attr_float` 采用字符串匹配提取出 `lat` 和 `lon`。

调整海拔解析逻辑

- **原理**：GPX 的海拔独立存在于 `<ele>` 标签中。
- **修改**：把原本处理 `coordinates` 冗长数字流的 `STATE_COORD_PARSE`，替换为了轻量的 `STATE_ELE_PARSE`。当读取到 `<ele>` 时，转入该状态收集数字，遇到下个 `<` 闭合时转换为浮点数并赋值给 `temp_point.alt`。

上下文和标签匹配更新

- 将 `Point` 替换为 `wpt`（对应你的检查点 / Waypoints）。
- 将 `LineString` / `Track` 替换为 `trkseg` 和 `trkpt`（对应航迹段和航迹点）。
- 自闭合标签支持：因为 GPX 经常出现没有高程的单行点（如 `<trkpt lat="x" lon="y" />`），状态机现在完美兼容这种 `is_self_closing` 格式，在提取完属性后会安全触发 `commit_point`，不漏点。

调用接口完全无痛兼容

- 函数签名依然是 `KmlError parse_kml_file(const char* input_file)`。
- 导出的 `point_temp.bin` 和 `line_temp.bin` 数据结构、分段回填数量（Total Segments）以及检查点 (CP) 的切割逻辑，都原封不动得到了保留。

代码

代码块

```
#include <stdio.h>
#include <stdint.h>
#include <string.h>
```

```c
#include <stdlib.h>
#include <float.h>
#include <ctype.h>
#include <math.h>
#include <cctype>

/* ================== 一、系统配置与常量定义 ================== */

#define READ_BUF_SIZE       4096
#define LINE_CACHE_SIZE     128
#define TAG_BUF_SIZE        64
#define NUM_BUF_SIZE        32
#define ATTR_BUF_SIZE       256     // 新增：用于容纳包含 lat 和 lon 的属性字符串

#define MAX_CP_COUNT        64
#define SPLIT_THRESHOLD     2e-4f
#define MIN_SEGMENT_COUNT   200
#define MAX_POINTS_PER_SEGMENT 3000

#define FILE_POINT_NAME     "D:\\KT_Project\\OPEN_SOURCE\\new\\X-TRACK-main\\Software\\X-Track\\USER\\App\\Resource\\Track\\point_temp.bin"
#define FILE_LINE_NAME      "D:\\KT_Project\\OPEN_SOURCE\\new\\X-TRACK-main\\Software\\X-Track\\USER\\App\\Resource\\Track\\line_temp.bin"
#define FILE_LINE_FILTERED_NAME     "D:\\KT_Project\\OPEN_SOURCE\\new\\X-TRACK-main\\Software\\X-Track\\USER\\App\\Resource\\Track\\line_temp_filter.bin"

/* ================== 二、数据结构定义 ================== */
typedef enum {
    KML_OK = 0,
    KML_ERR_FILE_OPEN,
    KML_ERR_IO_READ,
    KML_ERR_IO_WRITE,
    KML_ERR_SEEK,
    KML_ERR_BAD_FORMAT,
    KML_ERR_INTERNAL,
    KML_ERR_NO_DATA
} KmlError;

typedef enum {
    STATE_IDLE,
    STATE_TAG_NAME,
    STATE_ATTR_COLLECT,         // 新增：收集属性字符串 (lat/lon)
    STATE_TAG_END_NAME,
    STATE_ELE_PARSE             // 新增：解析 <ele> 标签内的数字
} ParseState;

typedef enum {
```

```
    STATE_CTX_ROOT,
    STATE_CTX_WPT,              // GPX: <wpt> 航点 (替代原 POINT)
    STATE_CTX_TRKSEG,           // GPX: <trkseg> 航段 (替代原 LINE)
    STATE_CTX_TRKPT             // GPX: <trkpt> 航迹点
} LogicState;

typedef struct {
    float lon;
    float lat;
    float alt;
} GeoPoint;

typedef struct {
    ParseState curr_state;
    LogicState logic_ctx;

    char tag_buf[TAG_BUF_SIZE];
    uint16_t tag_len;
    char attr_buf[ATTR_BUF_SIZE];// 新增: 属性缓冲区
    uint16_t attr_len;           // 新增: 属性长度
    char num_buf[NUM_BUF_SIZE];
    uint16_t num_len;

    GeoPoint temp_point;

    fpos_t line_len_pos;
    uint32_t line_point_cnt;

    GeoPoint line_cache[LINE_CACHE_SIZE];
    uint16_t cache_idx;

    FILE* fp_point;
    FILE* fp_line;

    KmlError error;

    uint32_t valid_point_cnt;

    GeoPoint cp_list[MAX_CP_COUNT];
    uint16_t cp_count;
    uint8_t cp_list_visit[MAX_CP_COUNT];

    fpos_t global_header_pos;
    fpos_t segment_header_pos;

    uint32_t total_segments;
    uint32_t curr_seg_points;
```

```c

} KmlCtx;

/* ================== 三、辅助函数实现 ================== */

static void reset_temp_point(KmlCtx* ctx) {
    ctx->temp_point.lon = 0.0f;
    ctx->temp_point.lat = 0.0f;
    ctx->temp_point.alt = 0.0f;
    ctx->num_len = 0;
    memset(ctx->num_buf, 0, NUM_BUF_SIZE);
}

static void flush_line_cache(KmlCtx* ctx) {
    if (ctx->cache_idx > 0) {
        size_t written = fwrite(ctx->line_cache, sizeof(GeoPoint), ctx->cache_idx, ctx->fp_line);
        if (written != ctx->cache_idx) {
            ctx->error = KML_ERR_IO_WRITE;
        }
        ctx->cache_idx = 0;
    }
}

static void close_current_segment(KmlCtx* ctx) {
    flush_line_cache(ctx);
    if (ctx->error != KML_OK) return;

    fpos_t end_pos;
    if (fgetpos(ctx->fp_line, &end_pos) != 0) { ctx->error = KML_ERR_SEEK; return; }

    if (fsetpos(ctx->fp_line, &ctx->segment_header_pos) != 0) { ctx->error = KML_ERR_SEEK; return; }
    if (fwrite(&ctx->curr_seg_points, sizeof(uint32_t), 1, ctx->fp_line) != 1) { ctx->error = KML_ERR_IO_WRITE; return; }

    if (fsetpos(ctx->fp_line, &end_pos) != 0) { ctx->error = KML_ERR_SEEK; return; }
}

static void start_new_segment(KmlCtx* ctx) {
    if (fgetpos(ctx->fp_line, &ctx->segment_header_pos) != 0) { ctx->error = KML_ERR_SEEK; return; }

    uint32_t placeholder = 0;
    if (fwrite(&placeholder, sizeof(uint32_t), 1, ctx->fp_line) != 1) {
```

```
        ctx->error = KML_ERR_IO_WRITE;
        return;
    }

    ctx->curr_seg_points = 0;
    ctx->total_segments++;
}

static int is_match_cp(KmlCtx* ctx, GeoPoint* p) {
    for (int i = 0; i < ctx->cp_count; i++) {
        if (ctx->cp_list_visit[i] == 1 || i == 1) continue;
        float d_lon = fabsf(p->lon - ctx->cp_list[i].lon);
        float d_lat = fabsf(p->lat - ctx->cp_list[i].lat);
        if (d_lon < SPLIT_THRESHOLD && d_lat < SPLIT_THRESHOLD) {
            printf("匹配第几个点:%d\n", i + 1);
            ctx->cp_list_visit[i] = 1;
            return 1;
        }
    }
    return 0;
}

/* ================== 四、核心逻辑：GPX 数据提取 ================== */

/* 解析属性字符串中的浮点数，兼容单引号和双引号 */
static float parse_attr_float(const char* attrs, const char* key) {
    char search_str[32];
    snprintf(search_str, sizeof(search_str), "%s=\"", key);
    const char* p = strstr(attrs, search_str);
    if (p) return strtof(p + strlen(search_str), NULL);

    snprintf(search_str, sizeof(search_str), "%s='", key);
    p = strstr(attrs, search_str);
    if (p) return strtof(p + strlen(search_str), NULL);

    return 0.0f;
}

static void commit_point(KmlCtx* ctx) {
    // 完整性检查：无经纬度视为无效
    if (ctx->temp_point.lon == 0.0f && ctx->temp_point.lat == 0.0f) {
        reset_temp_point(ctx);
        return;
    }

    if (ctx->logic_ctx == STATE_CTX_WPT) {
```

```c
        if (fwrite(&ctx->temp_point, sizeof(GeoPoint), 1, ctx->fp_point) != 1) {
            ctx->error = KML_ERR_IO_WRITE;
        }
        if (ctx->cp_count < MAX_CP_COUNT) {
            ctx->cp_list[ctx->cp_count++] = ctx->temp_point;
        }
    }
    else if (ctx->logic_ctx == STATE_CTX_TRKPT) {
        int need_split = is_match_cp(ctx, &ctx->temp_point);

        if (ctx->cache_idx < LINE_CACHE_SIZE) {
            ctx->line_cache[ctx->cache_idx++] = ctx->temp_point;
            ctx->curr_seg_points++;
        }
        else {
            ctx->error = KML_ERR_INTERNAL;
            return;
        }

        if (ctx->cache_idx >= LINE_CACHE_SIZE) flush_line_cache(ctx);

        if (need_split && ctx->curr_seg_points > MIN_SEGMENT_COUNT) {
            close_current_segment(ctx);
            if (ctx->error != KML_OK) return;
            start_new_segment(ctx);

            ctx->line_cache[ctx->cache_idx++] = ctx->temp_point;
            ctx->curr_seg_points++;
        }
    }

    reset_temp_point(ctx);
}

/* ================== 五、核心逻辑：XML 标签处理 ================== */

static void handle_tag_open(KmlCtx* ctx) {
    if (ctx->tag_len == 0) return;
    ctx->tag_buf[ctx->tag_len] = '\0';
    char* tag = ctx->tag_buf;

    if (tag[0] == '?' || tag[0] == '!') return;

    if (strcmp(tag, "wpt") == 0) {
        ctx->logic_ctx = STATE_CTX_WPT;
        reset_temp_point(ctx);
```

```
        ctx->temp_point.lat = parse_attr_float(ctx->attr_buf, "lat");
        ctx->temp_point.lon = parse_attr_float(ctx->attr_buf, "lon");
    }
    else if (strcmp(tag, "trkseg") == 0) {
        ctx->logic_ctx = STATE_CTX_TRKSEG;

        if (ctx->total_segments == 0) {
            if (fgetpos(ctx->fp_line, &ctx->global_header_pos) != 0) {
                ctx->error = KML_ERR_SEEK; return;
            }
            uint32_t zero = 0;
            if (fwrite(&zero, sizeof(uint32_t), 1, ctx->fp_line) != 1) {
                ctx->error = KML_ERR_IO_WRITE; return;
            }
        }
        start_new_segment(ctx);
    }
    else if (strcmp(tag, "trkpt") == 0) {
        ctx->logic_ctx = STATE_CTX_TRKPT;
        reset_temp_point(ctx);
        ctx->temp_point.lat = parse_attr_float(ctx->attr_buf, "lat");
        ctx->temp_point.lon = parse_attr_float(ctx->attr_buf, "lon");
    }
    else if (strcmp(tag, "ele") == 0) {
        if (ctx->logic_ctx == STATE_CTX_WPT || ctx->logic_ctx ==
STATE_CTX_TRKPT) {
            ctx->curr_state = STATE_ELE_PARSE;
            ctx->num_len = 0;
        }
    }
}

static void handle_tag_close(KmlCtx* ctx) {
    ctx->tag_buf[ctx->tag_len] = '\0';
    char* tag = ctx->tag_buf;

    if (strcmp(tag, "wpt") == 0) {
        commit_point(ctx);
        ctx->logic_ctx = STATE_CTX_ROOT;
    }
    else if (strcmp(tag, "trkpt") == 0) {
        commit_point(ctx);
        ctx->logic_ctx = STATE_CTX_TRKSEG;
    }
    else if (strcmp(tag, "trkseg") == 0) {
        if (ctx->curr_seg_points > 0) {
            close_current_segment(ctx);
```

```c
        }
        ctx->logic_ctx = STATE_CTX_ROOT;
    }
}

/* ================= 六、主解析循环入口 ================= */

KmlError parse_kml_file(const char* input_file) {
    KmlCtx* ctx = (KmlCtx*)malloc(sizeof(KmlCtx));
    if (!ctx) return KML_ERR_INTERNAL;
    uint8_t* read_buf = NULL;
    size_t n_read;
    int is_self_closing = 0;
    KmlError final_err = KML_OK;

    memset(ctx, 0, sizeof(KmlCtx));
    ctx->curr_state = STATE_IDLE;
    ctx->logic_ctx = STATE_CTX_ROOT;

    FILE* fp_in = fopen(input_file, "rb");
    if (!fp_in) return KML_ERR_FILE_OPEN;

    ctx->fp_point = fopen(FILE_POINT_NAME, "wb");
    ctx->fp_line = fopen(FILE_LINE_NAME, "wb");

    if (!ctx->fp_point || !ctx->fp_line) {
        ctx->error = KML_ERR_FILE_OPEN;
        goto cleanup;
    }

    read_buf = (uint8_t*)malloc(READ_BUF_SIZE);
    if (!read_buf) {
        ctx->error = KML_ERR_INTERNAL;
        goto cleanup;
    }

    while ((n_read = fread(read_buf, 1, READ_BUF_SIZE, fp_in)) > 0) {
        for (size_t i = 0; i < n_read; i++) {
            if (ctx->error != KML_OK) goto cleanup;

            char c = read_buf[i];

            // 专属模式：提取 <ele> 里的海拔数据
            if (ctx->curr_state == STATE_ELE_PARSE) {
                if (c == '<') {
                    ctx->num_buf[ctx->num_len] = '\0';
                    ctx->temp_point.alt = strtof(ctx->num_buf, NULL);
```

```c

                    ctx->curr_state = STATE_TAG_NAME;
                    ctx->tag_len = 0;
                    is_self_closing = 0;
                }
                else {
                    if (ctx->num_len < NUM_BUF_SIZE - 1) {
                        if ((c >= '0' && c <= '9') || c == '.' || c == '-' || c == '+' || c == 'e' || c == 'E') {
                            ctx->num_buf[ctx->num_len++] = c;
                        }
                    }
                }
                continue;
            }

            // XML 结构与属性解析模式
            switch (ctx->curr_state) {
            case STATE_IDLE:
                if (c == '<') {
                    ctx->curr_state = STATE_TAG_NAME;
                    ctx->tag_len = 0;
                    is_self_closing = 0;
                }
                break;

            case STATE_TAG_NAME:
                if (c == '>') {
                    ctx->tag_buf[ctx->tag_len] = '\0';
                    ctx->attr_buf[0] = '\0'; // 无属性
                    ctx->attr_len = 0;

                    if (is_self_closing) {
                        handle_tag_open(ctx);
                        handle_tag_close(ctx);
                    }
                    else {
                        handle_tag_open(ctx);
                    }

                    if (ctx->curr_state == STATE_TAG_NAME) { // 未被 ele 切换状态
                        ctx->curr_state = STATE_IDLE;
                    }
                }
                else if (isspace((int)c)) {
                    ctx->tag_buf[ctx->tag_len] = '\0';
                    ctx->curr_state = STATE_ATTR_COLLECT; // 切换去收集属性
```

```
                    ctx->attr_len = 0;
                }
                else if (c == '/') {
                    if (ctx->tag_len == 0) {
                        ctx->curr_state = STATE_TAG_END_NAME;
                    }
                    else {
                        is_self_closing = 1;
                    }
                }
                else {
                    if (ctx->tag_len < TAG_BUF_SIZE - 1) ctx->tag_buf[ctx->tag_len++] = c;
                }
                break;

            case STATE_ATTR_COLLECT:
                if (c == '>') {
                    ctx->attr_buf[ctx->attr_len] = '\0';

                    if (is_self_closing) {
                        handle_tag_open(ctx);
                        handle_tag_close(ctx);
                    }
                    else {
                        handle_tag_open(ctx);
                    }

                    if (ctx->curr_state == STATE_ATTR_COLLECT) {
                        ctx->curr_state = STATE_IDLE;
                    }
                }
                else if (c == '/') {
                    is_self_closing = 1;
                }
                else {
                    if (ctx->attr_len < ATTR_BUF_SIZE - 1) {
                        ctx->attr_buf[ctx->attr_len++] = c;
                    }
                }
                break;

            case STATE_TAG_END_NAME:
                if (c == '>') {
                    ctx->tag_buf[ctx->tag_len] = '\0';
                    handle_tag_close(ctx);
                    ctx->curr_state = STATE_IDLE;
```

```
                }
                else if (!isspace((int)c)) {
                    if (ctx->tag_len < TAG_BUF_SIZE - 1) ctx->tag_buf[ctx->tag_len++] = c;
                }
                break;

            default:
                ctx->curr_state = STATE_IDLE;
                break;
            }
        }
    }

    if (ferror(fp_in)) {
        ctx->error = KML_ERR_IO_READ;
        goto cleanup;
    }

    if (ctx->curr_seg_points > 0) {
        close_current_segment(ctx);
    }

    if (ctx->fp_line && ctx->error == KML_OK) {
        fpos_t end_pos;
        if (fgetpos(ctx->fp_line, &end_pos) == 0) {
            if (fsetpos(ctx->fp_line, &ctx->global_header_pos) == 0) {
                fwrite(&ctx->total_segments, sizeof(uint32_t), 1, ctx->fp_line);
                fsetpos(ctx->fp_line, &end_pos);
            }
            else {
                ctx->error = KML_ERR_SEEK;
            }
        }
        else {
            ctx->error = KML_ERR_SEEK;
        }
    }

cleanup:
    if (ctx) final_err = ctx->error;
    else final_err = KML_ERR_INTERNAL;

    if (read_buf) free(read_buf);
    if (fp_in) fclose(fp_in);

```

```c
    if (ctx) {
        if (ctx->fp_point) fclose(ctx->fp_point);
        if (ctx->fp_line) fclose(ctx->fp_line);
        free(ctx);
        ctx = NULL;
    }

    if (final_err != KML_OK) {
        remove(FILE_POINT_NAME);
        remove(FILE_LINE_NAME);
    }

    return final_err;
}

/* filter_kml_lines, verify_point_bin_stream, verify_line_bin_fixed, verify_line_bin_segmented 函数内容未变动，保持你原有实现即可（为了不占版面已省去，在应用中直接保留原代码即可） */

/*
* 对分段线段文件进行滤波，确保每个分段的点数不要超过阈值
*/
KmlError filter_kml_lines(const char* src_file_path, const char* dst_file_path) {
    FILE* fp_src = fopen(src_file_path, "rb");
    FILE* fp_dst = fopen(dst_file_path, "wb");
    KmlError ret_error = KML_OK;

    if (!fp_src || !fp_dst) {
        if (fp_src) fclose(fp_src);
        if (fp_dst) fclose(fp_dst);
        return KML_ERR_FILE_OPEN;
    }
    uint16_t write_idx;
    uint32_t total_segments = 0;
    //修改为堆上分配内存
    GeoPoint* read_cache = NULL;
    GeoPoint* write_cache = NULL;
    // 读取总段数
    if (fread(&total_segments, sizeof(uint32_t), 1, fp_src) != 1) {
        ret_error = KML_ERR_IO_READ;
        goto error_cleanup;
    }
    // 写入总段数
    if (fwrite(&total_segments, sizeof(uint32_t), 1, fp_dst) != 1) {
        ret_error = KML_ERR_IO_WRITE;
        goto error_cleanup;
```

```
    }

    read_cache = (GeoPoint*)malloc(sizeof(GeoPoint) * LINE_CACHE_SIZE);
    write_cache = (GeoPoint*)malloc(sizeof(GeoPoint) * LINE_CACHE_SIZE);
    if (!read_cache || !write_cache) {
        ret_error = KML_ERR_INTERNAL; // 内存不足
        goto error_cleanup;
    }
    /*
    // 栈上内存分配 (约 3KB)
    GeoPoint read_cache[LINE_CACHE_SIZE];
    GeoPoint write_cache[LINE_CACHE_SIZE];
    */
    write_idx = 0;

    // 逐段处理
    for (uint32_t i = 0; i < total_segments; i++) {
        uint32_t src_count = 0;
        if (fread(&src_count, sizeof(uint32_t), 1, fp_src) != 1) {
            ret_error = KML_ERR_IO_READ;
            goto error_cleanup;
        }

        // 记录回填位置
        fpos_t segment_head_pos;
        if (fgetpos(fp_dst, &segment_head_pos) != 0) {
            ret_error = KML_ERR_SEEK;
            goto error_cleanup;
        }

        // 写占位符
        uint32_t placeholder = 0;
        if (fwrite(&placeholder, sizeof(uint32_t), 1, fp_dst) != 1) {
            ret_error = KML_ERR_IO_WRITE;
            goto error_cleanup;
        }

        uint32_t dst_count = 0;
        write_idx = 0; // 每个新段开始时，重置写缓存索引

        // === 策略分支 ===
        if (src_count <= MAX_POINTS_PER_SEGMENT) {
            /* 场景 A: 直接复制 */
            uint32_t remain = src_count;
            while (remain > 0) {
                uint16_t chunk = (remain > LINE_CACHE_SIZE) ? LINE_CACHE_SIZE
: (uint16_t)remain;
```

```

                if (fread(read_cache, sizeof(GeoPoint), chunk, fp_src) != chunk) {
                    ret_error = KML_ERR_IO_READ;
                    goto error_cleanup;
                }

                // 统一使用 write_cache 写入
                if (write_idx + chunk > LINE_CACHE_SIZE) {
                    if (fwrite(write_cache, sizeof(GeoPoint), write_idx, fp_dst) != write_idx) {
                        ret_error = KML_ERR_IO_WRITE;
                        goto error_cleanup;
                    }
                    write_idx = 0;
                }

                memcpy(&write_cache[write_idx], read_cache, chunk * sizeof(GeoPoint));
                write_idx += chunk;
                remain -= chunk;
            }
            dst_count = src_count;
        }
        else {
            /* 场景 B: 均匀抽稀 */
            if (MAX_POINTS_PER_SEGMENT <= 1) {
                ret_error = KML_ERR_INTERNAL;
                goto error_cleanup;
            }

            float step = (float)(src_count - 1) / (float)(MAX_POINTS_PER_SEGMENT - 1);
            float next_sample_idx = 0.0f;
            uint32_t processed = 0;
            uint8_t is_last_point_processed = 0; // 新增: 标记尾点是否已处理
            while (processed < src_count) {
                uint32_t remain = src_count - processed;
                uint16_t chunk = (remain > LINE_CACHE_SIZE) ? LINE_CACHE_SIZE : (uint16_t)remain;

                if (fread(read_cache, sizeof(GeoPoint), chunk, fp_src) != chunk) {
                    ret_error = KML_ERR_IO_READ;
                    goto error_cleanup;
                }

```

```
                for (uint16_t k = 0; k < chunk; k++) {
                    uint32_t global_idx = processed + k;
                    int is_last_point = (global_idx == src_count - 1);

                    // 判定逻辑：保留当前点的条件
                    if (is_last_point || (float)global_idx >= next_sample_idx) {
                        // 核心修改：允许最多 MAX+1 个点
                        if (dst_count <= MAX_POINTS_PER_SEGMENT) {
                            write_cache[write_idx++] = read_cache[k];
                            dst_count++;
                            is_last_point_processed = is_last_point ? 1 : is_last_point_processed;
                            // 非尾点时推进采样索引
                            if (!is_last_point) {
                                next_sample_idx += step;
                            }
                        }

                        // 缓存满，刷盘
                        if (write_idx >= LINE_CACHE_SIZE) {
                            if (fwrite(write_cache, sizeof(GeoPoint), write_idx, fp_dst) != write_idx) {
                                ret_error = KML_ERR_IO_WRITE;
                                goto error_cleanup;
                            }
                            write_idx = 0;
                        }
                    }
                }
                processed += chunk;
            }
        }

        // 统一刷入剩余缓存 (无论场景A还是B)
        if (write_idx > 0) {
            if (fwrite(write_cache, sizeof(GeoPoint), write_idx, fp_dst) != write_idx) {
                ret_error = KML_ERR_IO_WRITE;
                goto error_cleanup;
            }
            write_idx = 0;
        }

        // 回填点数
        fpos_t end_pos;
```

```
        if (fgetpos(fp_dst, &end_pos) != 0) { ret_error = KML_ERR_SEEK; goto error_cleanup; }
        if (fsetpos(fp_dst, &segment_head_pos) != 0) { ret_error = KML_ERR_SEEK; goto error_cleanup; }
        if (fwrite(&dst_count, sizeof(uint32_t), 1, fp_dst) != 1) { ret_error = KML_ERR_IO_WRITE; goto error_cleanup; }
        if (fsetpos(fp_dst, &end_pos) != 0) { ret_error = KML_ERR_SEEK; goto error_cleanup; }
    }

    free(read_cache);
    free(write_cache);
    read_cache = NULL;
    write_cache = NULL;
    fclose(fp_src);
    fclose(fp_dst);
    fp_src = NULL; // 防止后续误用
    fp_dst = NULL;

    // 文件替换
    if (remove(src_file_path) != 0) {
        ret_error = KML_ERR_IO_WRITE; // 无法删除旧文件
        goto error_cleanup_no_src_dst;
    }
    if (rename(dst_file_path, src_file_path) != 0) {
        ret_error = KML_ERR_IO_WRITE;
        goto error_cleanup_no_src_dst;
    }

    return KML_OK;

error_cleanup:
    if (read_cache) free(read_cache);
    if (write_cache) free(write_cache);
    if (fp_src) fclose(fp_src);
    if (fp_dst) fclose(fp_dst);

error_cleanup_no_src_dst:
    remove(dst_file_path); // 失败时清理产生的临时文件
    return ret_error;
}



/**
 * @brief 验证Point文件（流式读取，不占用RAM）
```

```c
 */
void verify_point_bin_stream(const char* file_path) {
    printf("\n=== Checking Point File: %s ===\n", file_path);
    FILE* fp = fopen(file_path, "rb");
    if (!fp) { perror("Open failed"); return; }

    // 1. 获取文件大小和总点数
    fseek(fp, 0, SEEK_END);
    long file_size = ftell(fp);
    size_t total_points = file_size / sizeof(GeoPoint);
    printf("Total Points: %zu (Size: %ld bytes)\n", total_points, file_size);

    if (total_points == 0) { fclose(fp); return; }

    GeoPoint pt;
    rewind(fp);

    // 2. 打印
   // printf("--- Head (First 10) ---\n");
    for (size_t i = 0; i < total_points; i++) {
        fread(&pt, sizeof(GeoPoint), 1, fp);
        printf("[%zu] %.6f, %.6f, %.2f\n", i + 1, pt.lon, pt.lat, pt.alt);
    }
    fclose(fp);
}

/**
 * @brief 打印单个点辅助函数
 */
static void print_point(const char* label, size_t index, GeoPoint p) {
    printf("%s[%zu] Lon: %.6f, Lat: %.6f, Alt: %.2f\n", label, index, p.lon, p.lat, p.alt);
}

/**
 * @brief 验证 Line 文件 (新格式: Header + Array)
 * 格式: [TotalCount (4B)] + [GeoPoint 1] + [GeoPoint 2] ...
 */
void verify_line_bin_fixed(const char* file_path) {
    printf("\n=== Checking Line File (Header Mode): %s ===\n", file_path);

    FILE* fp = fopen(file_path, "rb");
    if (!fp) {
        perror("Error opening file");
        return;
    }

```

```c
    // 1. 读取文件头的总点数
    uint32_t total_points = 0;
    if (fread(&total_points, sizeof(uint32_t), 1, fp) != 1) {
        printf("Error: File too short, cannot read header.\n");
        fclose(fp);
        return;
    }

    printf("Header says Total Points: %u\n", total_points);

    // (可选) 校验文件实际大小是否匹配
    // 理论大小 = 4字节头 + 点数 * 12字节
    long expected_size = sizeof(uint32_t) + (long)total_points * sizeof(GeoPoint);
    fseek(fp, 0, SEEK_END);
    long actual_size = ftell(fp);
    if (actual_size != expected_size) {
        printf("Warning: File size mismatch! Expected %ld bytes, got %ld bytes.\n", expected_size, actual_size);
        // 注意: 如果不匹配, 我们仍然尝试读取, 但在生产环境中这通常意味着文件损坏
    }

    // 如果没有点, 直接退出
    if (total_points == 0) {
        fclose(fp);
        return;
    }

    // 2. 打印前 10 个点
    // 文件指针现在正好位于 Header 之后 (偏移量 4), 直接读取即可
    fseek(fp, sizeof(uint32_t), SEEK_SET); // 确保指针在数据区开头

    printf("--- Head (First 10) ---\n");
    size_t count_to_read = (total_points < 10) ? total_points : 10;
    GeoPoint pt;

    for (size_t i = 0; i < count_to_read; i++) {
        fread(&pt, sizeof(GeoPoint), 1, fp);
        print_point("H", i, pt);
    }

    // 3. 打印后 10 个点
    if (total_points > 10) {
        printf("--- Tail (Last 10) ---\n");

        // 计算偏移量: Header大小 + (总点数 - 10) * 点大小
```

```c
        long offset = sizeof(uint32_t) + (long)((total_points - 10) * sizeof(GeoPoint));

        // 直接跳转到倒数第10个点的位置
        fseek(fp, offset, SEEK_SET);

        for (size_t i = 0; i < 10; i++) {
            // 读取并打印
            if (fread(&pt, sizeof(GeoPoint), 1, fp) == 1) {
                // 计算该点的全局索引
                size_t global_idx = total_points - 10 + i;
                print_point("T", global_idx, pt);
            }
        }
    }

    fclose(fp);
}

void verify_line_bin_segmented(const char* file_path) {
    printf("\n=== Checking Segmented Line File: %s ===\n", file_path);
    FILE* fp = fopen(file_path, "rb");
    if (!fp) {
        printf("Error: Cannot open file %s\n", file_path);
        return;
    }

    // 1. 读取总段数
    uint32_t total_segs = 0;
    if (fread(&total_segs, sizeof(uint32_t), 1, fp) != 1) {
        printf("Error: Failed to read total segments\n");
        fclose(fp);
        return;
    }
    printf("Total Segments: %u\n", total_segs);

    // 2. 遍历每一段
    for (uint32_t s = 0; s < total_segs; s++) {
        uint32_t pts_in_seg = 0;
        if (fread(&pts_in_seg, sizeof(uint32_t), 1, fp) != 1) {
            printf("Error: Failed to read segment %u point count\n", s + 1);
            fclose(fp);
            return;
        }
        printf("  Segment %u: %u points\n", s + 1, pts_in_seg);

        // 跳过点数据，或者打印首尾点
```

```c
        if (pts_in_seg > 0) {
            GeoPoint p;
            if (fread(&p, sizeof(GeoPoint), 1, fp) != 1) {
                printf("Error: Failed to read start point of segment %u\n", s + 1);
                fclose(fp);
                return;
            }
            printf("    Start: %.6f, %.6f\n", p.lon, p.lat);

            // 跳过中间的点
            if (pts_in_seg > 1) {
                if (fseek(fp, (pts_in_seg - 2) * sizeof(GeoPoint), SEEK_CUR) != 0) {
                    printf("Error: Failed to seek in segment %u\n", s + 1);
                    fclose(fp);
                    return;
                }
                if (fread(&p, sizeof(GeoPoint), 1, fp) != 1) {
                    printf("Error: Failed to read end point of segment %u\n", s + 1);
                    fclose(fp);
                    return;
                }
                printf("    End  : %.6f, %.6f\n", p.lon, p.lat);
            }
        }
    }
    fclose(fp);
}

```

使用说明

基础数据单元

`GeoPoint` 结构体

代码中所有的坐标点都以这个结构体形式存储。

- **大小**：12 字节 (3 * 4 bytes)
- **内存布局**：

代码块

```
|   0-3 bytes   |   4-7 bytes   |   8-11 bytes  |
|---------------|---------------|---------------|
|  float (Lon)  |  float (Lat)  |  float (Alt)  |
```

点数据文件 (`point_temp.bin`)

该文件用于存储独立的 `<Point>` 标签数据（如地标）。

- **结构类型：纯数组结构 (Flat Array)**
- **特点**：没有任何文件头或元数据，文件中全是紧密排列的 `GeoPoint` 数据。
- **文件布局图**：

代码块

```
[ Start of File ]
+---------------------------+
| GeoPoint 1 (12 Bytes)     |
+---------------------------+
| GeoPoint 2 (12 Bytes)     |
+---------------------------+
| ...                       |
+---------------------------+
| GeoPoint N (12 Bytes)     |
+---------------------------+
[ End of File ]
```

线/轨迹数据文件 (`line_temp.bin`)

该文件用于存储 `<LineString>` 或 `<Track>` 数据。由于轨迹可能由多条不连续的线段组成（分段），该文件采用了 **分段存储结构**。

- **结构类型：层级头结构 (Hierarchical Header)**
- **逻辑流程**：先读总段数 -> 循环 -> 读当前段点数 -> 读具体坐标。

文件布局详细说明：

偏移量 (Offset)	数据类型	描述
0x00	uint32_t	**总分段数 (Total Segments)**，假设值为 N
0x04	uint32_t	**第 1 段的点数 (Count 1)**
0x08	GeoPoint	第 1 段 - 第 1 个点
...	...	...
...	GeoPoint	第 1 段 - 最后一个点
Next	uint32_t	**第 2 段的点数 (Count 2)**
Next+4	GeoPoint	第 2 段 - 第 1 个点
...	...	...
Next	uint32_t	**第 N 段的点数 (Count N)**
...	GeoPoint	第 N 段 - 坐标数据

可视化结构图：

代码块

```
[ Global Header ]
+-----------------------------------+
| Total Segments (4 Bytes)          |  <-- 例如: 2 (表示有两条独立的轨迹)
+-----------------------------------+

[ Segment 1 Block ]
+-----------------------------------+
| Segment 1 Point Count (4 Bytes)   |  <-- 例如: 1000
+-----------------------------------+
| GeoPoint 1-1 (12 Bytes)           |
| GeoPoint 1-2 (12 Bytes)           |
| ...                               |
| GeoPoint 1-1000 (12 Bytes)        |
+-----------------------------------+

[ Segment 2 Block ]
+-----------------------------------+
| Segment 2 Point Count (4 Bytes)   |  <-- 例如: 500
+-----------------------------------+
| GeoPoint 2-1 (12 Bytes)           |
```

```
| ...                                |
| GeoPoint 2-500 (12 Bytes)          |
+-----------------------------------+

[ End of File ]
```

调用说明

代码块

```
const char* kml_file_path = "D:\\KT_Project\\OPEN_SOURCE\\new\\X-TRACK-main\\Software\\X-Track\\USER\\App\\Resource\\Track\\guangzhou100.gpx";  // 你的KML文件路径

    printf("Start parsing...\n");
    KmlError err = parse_kml_file(kml_file_path);//提取标记点和航线轨迹到，并按标记点切分到
                                                 //FILE_LINE_NAME文件
    if (err == KML_OK) {
        printf("Success. Data extracted to .bin files.\n");
    }
    else {
        printf("Failed with error code: %d\n", err);
    }
    //对FILE_LINE_NAME文件分段文件进行滤波处理确保每段的点数不超过MAX_POINTS_PER_SEGMENT+1个点
    filter_kml_lines(FILE_LINE_NAME, FILE_LINE_FILTERED_NAME);
    verify_line_bin_segmented(FILE_LINE_NAME);//验证可视化打印输出
    verify_point_bin_stream(FILE_POINT_NAME);//验证可视化打印输出
```

调试说明，**先坚持拿来主义，不去关注内部的代码实现**，按照调用说明先调用**parse_kml_file(kml_file_path)**

再调用**filter_kml_lines(FILE_LINE_NAME, FILE_LINE_FILTERED_NAME)**即可。